SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I **Lekkala Dattu Kumar(192324141)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: L Dattu Kumar

Annexure A

Scenario-based

Question 1: Secure Data Transmission Using Block Cipher

Scenario: A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

A. Explain why this issue occurs.

1. Introduction

A block cipher is a type of encryption algorithm that converts plaintext (original data) into ciphertext (encrypted data) using a secret key.

The original data is divided into fixed-size blocks, and each block is encrypted separately or according to a specific operating mode.

Examples of block cipher algorithms include AES, DES, and 3-DES.

In the given scenario, the company uses Electronic Codebook (ECB) mode to encrypt repeated financial data. ECB mode can reveal patterns because it encrypts identical plaintext blocks into identical ciphertext blocks when the same key is used.

2. How ECB mode works

ECB mode encrypts every plaintext block independently using the same secret key.

The encryption formula is:

$$ C_i=E_K(P_i) $$

Where:

- P_i = Plaintext block
- C_i = Ciphertext block
- E_K = Encryption algorithm using key K

Diagram: ECB encryption process

Plaintext blocks

Block A

Block B

Block A

Same secret key K

Encrypt

Encrypt

Encrypt

Ciphertext blocks

X

Y

X

Repeated plaintext produces repeated ciphertext.

3. Why does ECB reveal patterns?

ECB has the following weaknesses:

1. Identical plaintext blocks: The same plaintext block produces the same ciphertext block under the same key.
2. Pattern visibility: An attacker can identify repeated blocks in the encrypted data.
3. Information leakage: The attacker may learn that certain financial records or fields are repeated, even without knowing their exact contents.
4. No chaining: Each block is encrypted independently, so one block does not depend on the previous block.
5. No integrity protection: ECB does not detect whether ciphertext has been modified or rearranged.

Example

Suppose a company sends the following financial records:

Plaintext block	**ECB ciphertext**
Payment = ₹5,000	A7F2
Payment = ₹8,000	B4C9
Payment = ₹5,000	A7F2

The repeated payment block produces the same ciphertext. An attacker can observe the repetition and identify patterns in the data.

Note: These ciphertext values are illustrative, not actual encryption outputs.

4. Impact on the company

The ECB weakness can cause:

- Leakage of repeated transaction patterns.
- Reduced confidentiality of financial information.
- Greater opportunity for attackers to analyze encrypted records.
- Potential manipulation or rearrangement of encrypted blocks if the application does not provide integrity protection.

Therefore, ECB is not suitable for encrypting repeated financial data that requires strong confidentiality.

B. Suggest a more secure block cipher mode and justify your choice.

Recommended mode: CBC (Cipher Block Chaining)

The company can replace ECB with Cipher Block Chaining (CBC) mode when using a suitable block cipher such as AES.

CBC combines each plaintext block with the previous ciphertext block before encryption. It uses an Initialization Vector (IV) for the first block.

The encryption formula is:

\[C_1=E_K(P_1\oplus IV) \]

For subsequent blocks:

\[C_i=E_K(P_i\oplus C_{i-1}) \]

Where:

- \(C_i\) = Current ciphertext block
- \(P_i\) = Current plaintext block
- \(C_{i-1}\) = Previous ciphertext block
- \(IV\) = Initialization Vector
- \(\oplus\) = XOR operation

Diagram: CBC encryption process

Plaintext block P_1

XOR with IV

AES encryption using K

Ciphertext C_1

Plaintext block P_2

XOR with previous C_1

AES encryption using K

Ciphertext C_2

Each plaintext block is combined with the previous ciphertext block before encryption.

Advantages of CBC mode

1. Hides repeated patterns: Identical plaintext blocks can produce different ciphertext blocks because of the chaining process.
2. Improves confidentiality: It makes direct pattern analysis more difficult than in ECB.
3. Uses an IV: A fresh, unpredictable IV should be used for each encryption under the same key.
4. Suitable for block encryption: It can encrypt financial records when correctly implemented with appropriate padding and key management.

Comparison between ECB and CBC

Feature	**ECB mode**	**CBC mode**
Encryption method	Each block independently	Each block depends on the previous ciphertext
Repeated plaintext	Produces repeated ciphertext	Usually produces different ciphertext with a fresh IV
Pattern visibility	High	Reduced
Initialization Vector	Not required	Required

Error propagation	Errors affect the corresponding block	Errors can affect the current and following decrypted block
Integrity protection	Not provided	Not provided
Suitability for financial data	Not recommended	Better confidentiality than ECB

Important security consideration

CBC improves confidentiality but does not provide integrity or authentication. For a modern banking system, AES-GCM is generally a better choice because it provides both encryption and authentication. If CBC is used, it should be combined with a secure message authentication mechanism and correct IV and padding handling.

Conclusion

ECB reveals repeated patterns because identical plaintext blocks produce identical ciphertext blocks. CBC reduces this problem by chaining blocks together using an IV and previous ciphertext. However, for modern financial transactions, AES-GCM is the preferred choice because it protects both the confidentiality and integrity of the data.

Question 5: Cryptography for IoT Devices

Scenario: A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analyze the constraints of IoT devices.

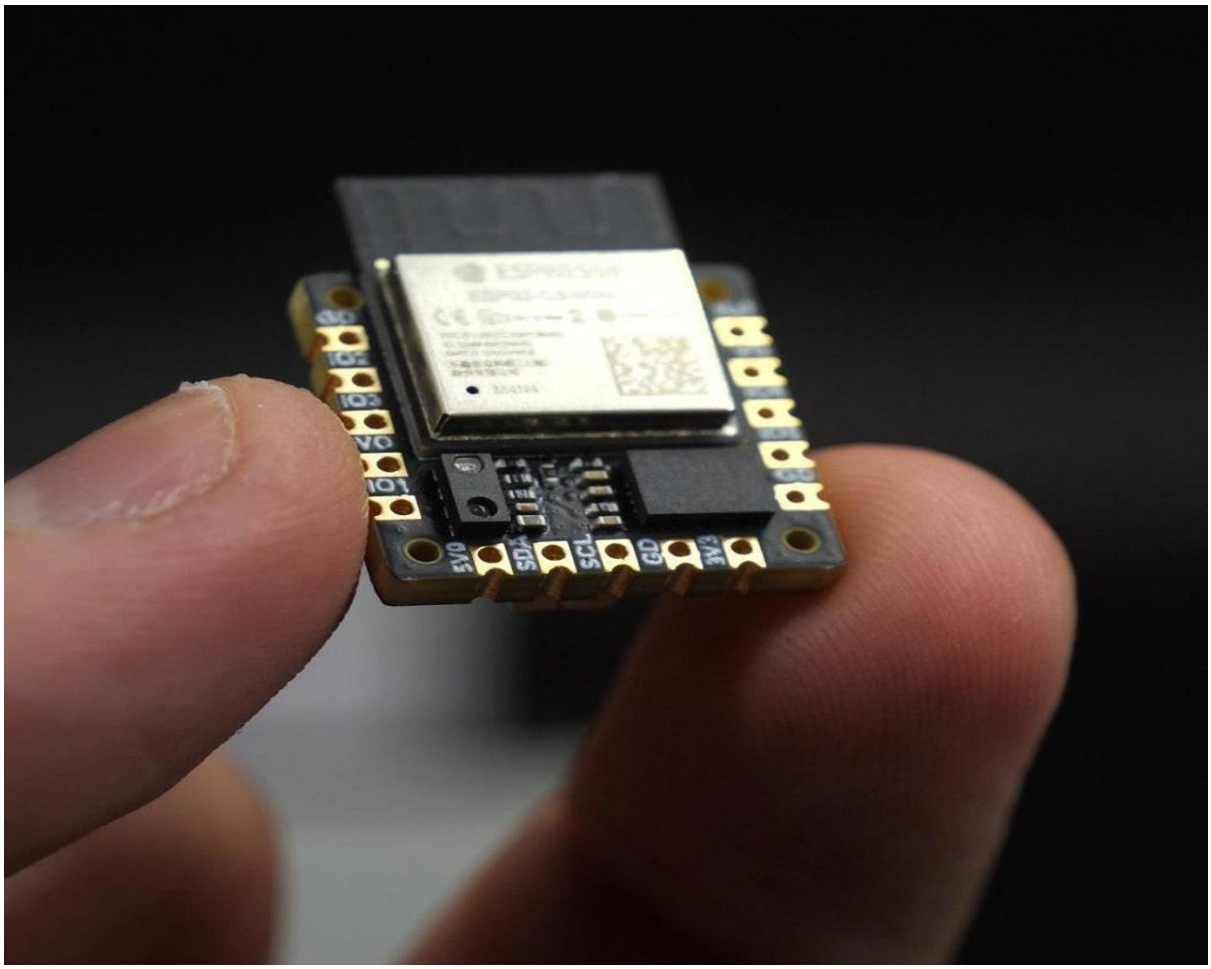
1. Introduction

The Internet of Things (IoT) connects physical devices such as smart locks, temperature sensors, smart lights, cameras, and motion detectors to a network.

These devices exchange information over wired or wireless connections. Because they may transmit sensitive information, cryptography is needed to protect their communication.

However, many IoT devices have limited processing power, memory, battery capacity, and communication bandwidth. Therefore, the designer must choose a cryptographic method that provides strong security without consuming too many resources.

2. Major constraints of IoT devices



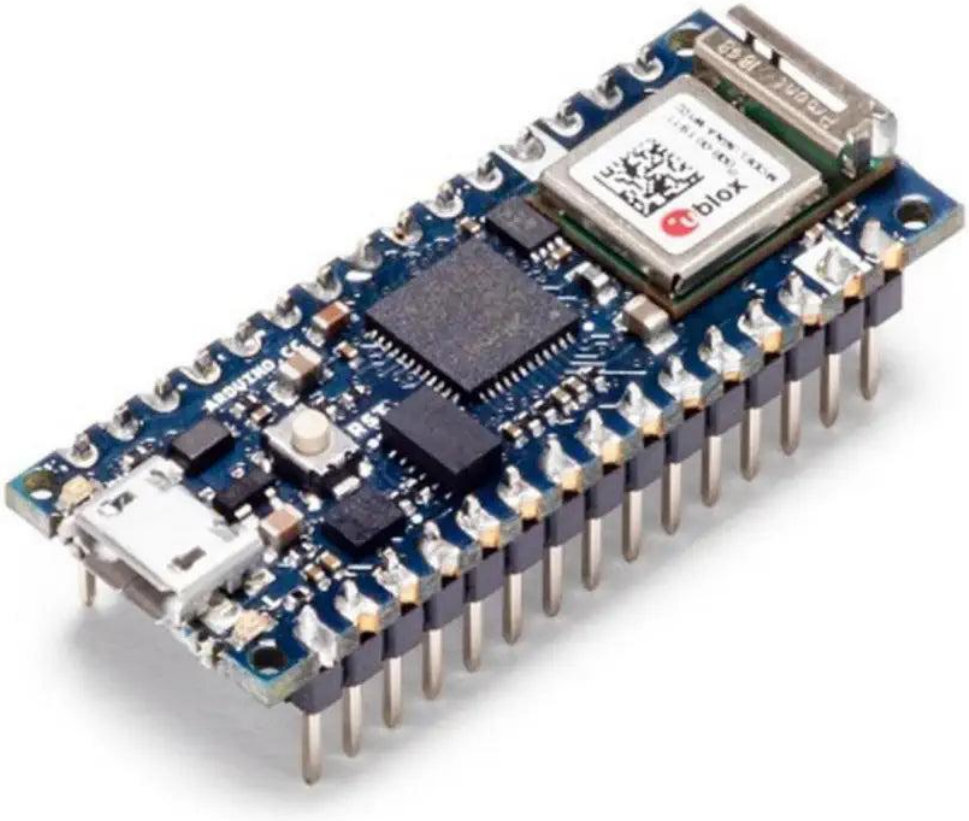
1. Limited processing power

IoT devices often use small microcontrollers. Complex cryptographic calculations can take more time and processing power.



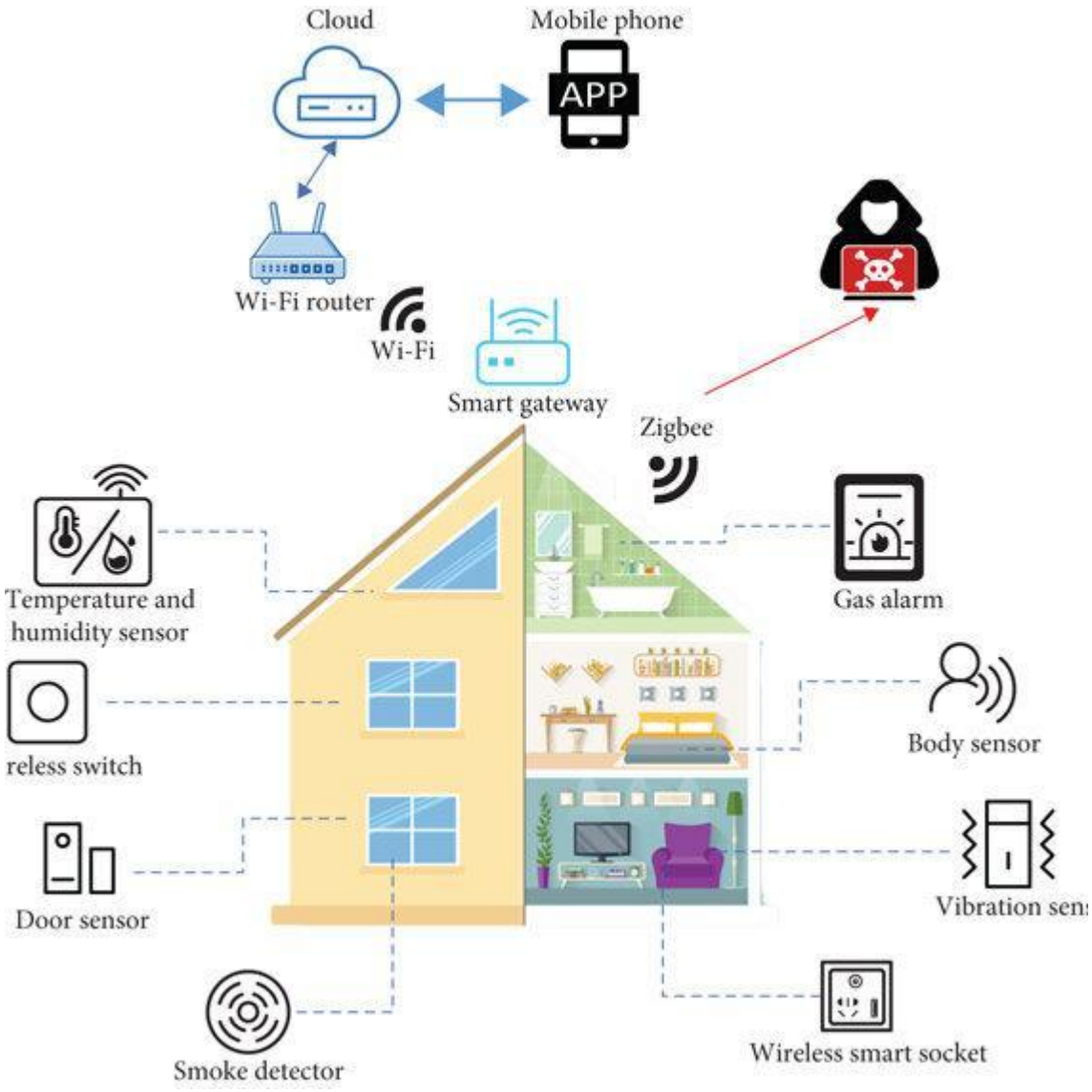
2. Limited battery power

Many sensors operate on batteries. Heavy calculations increase energy consumption and may reduce battery life.



3. Limited memory and storage

Small devices cannot always store large keys, certificates, or complex cryptographic software.



4. Limited bandwidth

IoT devices may use low-bandwidth wireless networks. Large public keys and certificates can increase communication overhead.



5. Strong security requirements

Smart locks, cameras, and sensors must prevent unauthorized access, impersonation, and modification of messages.

3. Why is the algorithm choice important?

A suitable cryptographic algorithm should:

- Provide strong security.
- Require less processing time.
- Consume less battery power.
- Use less memory and bandwidth.
- Support secure key establishment and device authentication.

RSA and ECC can both support public-key cryptography, but their resource requirements differ.

B. Recommend a suitable cryptographic approach and justify your answer.

Recommended approach: ECC

Elliptic Curve Cryptography (ECC) is generally more suitable than RSA for low-power IoT devices because it provides comparable security with much smaller keys.

ECC is based on mathematical operations involving points on elliptic curves over finite fields.

RSA relies on the difficulty of factoring large integers, while ECC relies on the difficulty of the elliptic-curve discrete logarithm problem.

1. RSA versus ECC

Feature	**RSA**	**ECC**
Key size for roughly 128-bit security	3072-bit RSA	256-bit ECC
Processing requirements	Can be resource-intensive, especially for private-key operations	Often more efficient for comparable security
Memory requirements	Larger keys and associated data	Smaller keys and associated data
Communication overhead	Generally higher	Generally lower
Battery consumption	Can be higher, depending on implementation	Often lower for comparable security
Suitability for low-power IoT	Less suitable in many constrained devices	Generally more suitable
Security	Strong when correctly implemented	Strong when correctly implemented

The key sizes shown are approximate examples of commonly accepted security-strength equivalence, not a guarantee of identical performance on every device.

2. Diagram: ECC-based secure communication in a smart home

IoT Sensor

Creates an ECC key pair

Smart Home Gateway

Creates an ECC key pair

Exchange and authenticate public keys

For example, using ECDH with authenticated public keys

Shared secret established

Both devices derive the same secret without sending it directly.

Secure data transmission

Use a derived symmetric key with authenticated encryption, such as AES-GCM or ChaCha20-Poly1305.

3. How ECC works in the smart home system

Step 1: Key generation

Each IoT device generates a private key and a corresponding public key.

- The private key must remain secret.
- The public key can be shared with the other device.

Step 2: Public-key exchange

The sensor and gateway exchange their public keys. The public keys must be authenticated, for example, through certificates or a trusted device-pairing process.

Step 3: Shared-secret generation

The devices use Elliptic Curve Diffie–Hellman (ECDH) to independently calculate the same shared secret.

The secret itself is never sent over the network.

Step 4: Session-key derivation

Both devices use a key derivation function to generate a symmetric session key from the shared secret.

Step 5: Secure communication

The devices encrypt and authenticate their messages using the session key. This protects the information against unauthorized reading and tampering.

4. Real-time example

Consider a smart door lock connected to a home gateway.

1. The door lock and gateway establish an authenticated ECC-based connection.
2. They use ECDH to establish a shared secret.
3. Both devices derive a session key.
4. When the owner unlocks the door using an authorized application, the command is transmitted through an authenticated encrypted channel.
5. The gateway and lock verify the message's authenticity before accepting the command.

This approach reduces cryptographic communication overhead while helping protect sensitive smart-home operations.

5. Advantages of ECC for IoT

- Smaller keys: Reduces storage and communication requirements.
- Lower resource use: Often requires less computation and energy for comparable security, depending on the implementation.
- Strong security: Provides strong protection when modern curves and secure implementations are used.
- Efficient key exchange: ECDH can establish a shared secret for secure communication.
- Scalability: Works well for networks containing many constrained devices.

6. Limitations and precautions

ECC still requires careful implementation. The designer should:

- Use established, secure cryptographic libraries.
- Protect private keys from extraction.
- Authenticate public keys to prevent man-in-the-middle attacks.
- Use secure random-number generation.
- Apply session-key rotation and secure firmware updates.
- Use authenticated encryption to protect both confidentiality and integrity.

ECC is not automatically secure merely because it uses smaller keys; correct implementation and key management are essential.

Conclusion

ECC is generally the better choice for low-power IoT devices because it provides strong security with smaller keys and lower communication overhead than RSA at comparable security levels. An IoT system can use ECC-based authenticated key exchange, such as ECDH, to establish a shared secret, followed by symmetric authenticated encryption for everyday communication. This approach balances security, performance, memory usage, and battery life in a smart home system.

RUBRICS FOR CALCULATION:

Criteria	**Weighta ge (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

Decision Making	20%	Makes well reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand